

DM571 - Receipt Management System

moami14, tkhan23, jogas23, kaami23

02/12/2025

Contents

1	Introduction	3
2	Part 1 - Discussion	3
2.1	Assumptions	3
2.2	Classification of the different stakeholders	3
2.2.1	Saliency Model	3
2.3	A description and classification of the different users	5
2.4	Prioritized backlog	7
2.4.1	Functional Requirements	7
2.4.2	Non-functional requirements	9
2.4.3	Discussion on backlog structure	9
2.4.4	Minimum Viable Product (MVP)	10
3	Part 2 - Implementation	11
3.1	Users and Receipts	11
3.2	Implementation details	12
3.2.1	Implemented functionality	12
3.3	User Interface Design	13
3.3.1	Demonstration video	17
3.4	UML	17
3.5	Sequence Diagram - Creating a Receipt	19
3.6	Sequence Diagram - Manager Approves a Receipt	20
4	System Architecture - C4 Model	21
4.1	C4 Model	21
4.1.1	System Context Diagram	21
4.1.2	Container Diagram	22
4.1.3	Component Diagram	22

4.1.4	Code Diagram	23
4.2	3 Layered Arcitecture	24
4.3	Renzel's Distribution Patterns	25
4.4	Agile Principles	25
4.5	Unit Testing	26
4.5.1	Coverage	26
4.5.2	Unit Test Coverage Report	27
4.6	Mobile API	28
4.7	Cyclomatic Complexity	31
4.8	Bank integration	35
A	Credit - Who Wrote What	36
B	System Source Code	36
B.1	enums.py	36
B.2	database.py	36
B.3	models.py	37
B.4	user_application.py	39
B.5	receipt_application.py	40
B.6	receipt_factory.py	42
B.7	user_factory.py	43
B.8	app.py	43
C	Mobile API Specification (OpenAPI)	48

1 Introduction

The purpose of this report is to present the design and implementation of the Receipt Management System developed for the DM571 project. The system supports the full reimbursement workflow used at GM, where salesmen submit receipts, Accountants handle them, Managers approve or reject them, and Administrators oversee the platform. The report is divided into three major parts. Part 1 covers the analysis phase, including stakeholders, user roles, assumptions, and the prioritized backlog. Part 2 focuses more on the technical implementation of the MVP and explains the architecture, testing, and API specification. Part 3 covers our system integration with external services.

2 Part 1 - Discussion

2.1 Assumptions

Before defining the system requirements, we need to make assumptions to improve our understanding of how the system is expected to function. These assumptions are not drawn from the interview, but from missing information and chosen design choices.

- All users belong to the same organization (GM) and are created inside our system, not via external providers.
- Each Salesman has exactly one Manager responsible for approving their receipts.
- The current MVP only supports a web-based interface. The mobile API is used by a future mobile app, but not implemented yet.
- Data is stored in memory only. If the server restarts, users and receipts are lost
- Fraud cases and edge cases are rare, and the main focus is on the normal flows.

2.2 Classification of the different stakeholders

2.2.1 Salience Model

Dormant Stakeholders Dormant stakeholders have high power, but are usually not very involved. They could be senior executives of GM or any

parent companies of GM. This gives them power to change or even shut-down the project. These stakeholders are not very active (hence the name “dormant”), but their interest can change and it may have an impact on the project. These senior executives are most likely in another department due to the low legitimacy. These stakeholders don’t require a lot of attention, but it would not be a bad idea to keep them update occasionally so that they are not caught by surprise.

Demanding Stakeholders These stakeholders have high urgency. The interview mentions how we need to log data to combat fraud. Some sales-people might feel that this is intrusive, and could voice concerns over this, but they lack the power to change anything. Labour unions or other employee groups could also be potential demanding stakeholders if they lack the power to change anything. However, given the nature of the data, we are collecting (about the receipts) we do not think that this will be an issue.

Discretionary Stakeholders These stakeholders have high legitimacy. They have a valid reason to be involved/interested in the project, but they don’t have the power to change anything, nor are their needs urgent. An example could be some sort of (internal) financial auditors who are interested in the receipts.

Dominant Stakeholders These stakeholders have high power and legitimacy. Since we are logging data, then regulatory units could be stakeholders, since they might have non-urgent requirements for how to handle this data. Not only do they have power to impact the project, but they also have a valid, legal right to do so (high legitimacy). They could easily become a core stakeholder if there arises some urgency. Maybe they give us a deadline to fix some data-related issues or something similar. This can have a big impact on the project.

There could also be dominant stakeholders from the company itself. Both the finance and IT departments could have their own concerns in regard to different requirements for the project. This is especially the case for the finance department, since it is their rightful role to be concerned with the issue of handling receipts.

Dangerous Stakeholders These stakeholders have high power and high urgency. They don’t have a legitimate claim to the system. Generic examples can include protest groups or media outlets that are trying to get a story. While one must be careful around these, we do not think that there are

any dangerous stakeholders in the given case. Perhaps if the data logging becomes too intrusive, media outlets could become a dangerous stakeholder once they get a hold of the story.

Dependent Stakeholders These stakeholders have high urgency and legitimacy. An example of this could be the salespeople (the users), since their concerns with the system are valid, but they do not have the power to change anything. These stakeholders might go to the dominant stakeholders to get their voices heard if they are not managed properly.

Core Stakeholders Core stakeholders are the most important stakeholders. They are usually very engaged with the project and expect to be consulted regularly. This could for example be the finance department leadership, who are in charge of overseeing the project for the company. Their power is high, they have a legitimate reason to be involved in the project and their needs are usually urgent due to the high engagement. These stakeholders cannot be ignored, and one must keep them engaged.

2.3 A description and classification of the different users

The system is intended for several user groups, each with their own responsibilities and expectations. These users can be categorized as: End users, Super users, and System users. The primary user base consists of Salesmen, Accountants, Managers, and Administrators. Each group benefits from reduced manual work, improved transparency, and a structured reimbursement flow.

The End users of the system are the Salesmen. The Salesmen travel frequently and generate most of the company's receipts, while also needing their reimbursement. Their primary need is a simple way to submit receipts while traveling. The system gives them a simple and easy way to upload receipts, reducing the risk of losing receipts and allowing for faster reimbursement.

The Super users consist of accountants and managers. Accountants are responsible for validating receipts before they can be approved. The system gives them a structured overview of all submissions, which reduces cases of duplicate handling, missing information, and manual errors. The system also provides logs that record who submitted, reviewed, or edited a receipt, which helps avoid fraud. Managers, who make the final approval or rejections, gain a clear and consistent interface that presents all relevant information, allow-

ing them to make quick decisions. The system also prevents managers from approving their own receipts, which upholds integrity and supports fraud prevention.

The System users are the administrators, who oversee the technical and operational aspects of the system. Administrators maintain accounts, roles, and general system functionality. They benefit from centralized tools that make managing users easier, help troubleshoot issues through system logs, and improve the efficiency of support. This reduces the reliance on the development team and keeps the system running smoothly.

Daily maintenance and user support will be carried out mainly by the administrators. They are responsible for creating accounts, updating permissions, solving login issues, reviewing system logs, and assisting users. For more complex issues, the development team will provide support.

The system interacts with both internal and external components. Internally, it integrates with modules in charge of user and role management, receipt submission, approval, and system logging. Externally, the system communicates with a mobile application through API endpoints. Additionally, the system integrates with the external bank API (finance.lutzen.dk), which allows it to fetch payment data and update transaction statuses.

2.4 Prioritized backlog

2.4.1 Functional Requirements

ID	User Story	Purpose	SP
F1	As a Salesman, I want to upload a receipt so that I can get reimbursed	The core function of the entire system	5
F2	As a Salesman, I want to see the status of my receipts so that I know when they are approved	Improves transparency between Salesmen and Accountants	3
F3	As an Accountant, I want to check each receipt and mark it as either correct or needing changes, so that only accurate receipts move on to approval the same day	Ensures that only accurate and complete receipts continue in the workflow	5
F4	As a Manager, I want to approve or reject validated receipts within the system, so that reimbursements can be completed within the same approval cycle	Final step in the business workflow	5
F5	As an Admin, I want to create, update, and deactivate user accounts and assign specific roles, so that all users have correct access rights before each new user begins using the system	The system cannot function without proper user role management	5
F6	As an Accountant, I want the system to automatically indicate if a receipt is a duplicate, so that I can avoid working twice and ensure each receipt is handled only once	Prevents duplicate handling and fraud	8

ID	User Story	Purpose	SP
F7	As the System, I must record who submitted, validated, edited, or approved each receipt, including timestamps, so that all actions can be tracked for further investigation	Supports auditing and fraud prevention	8
F8	As an Admin or Accountant, I want to view all receipts in the system, so that I can quickly find the one I need to review or assist with	Enables efficient validation, approval, and troubleshooting	3
F9	As an Admin, I want to see all users and their submitted receipts, so that I can help them when something is stuck or unclear	Supports user assistance and maintains system workflow	5
F10	As a Manager, I want to add comments when rejecting a receipt, so that the Salesman understands what needs to be corrected	Improves communication and prevents confusion	2
F11	As a User, I want the system to work well on mobile devices, so that I can use it easily while traveling	Salesmen often work remotely	2
F12	As a Mobile App, I need an API to fetch a user's receipts, so that the user can see their submissions on their device	Allows users to view receipts in real time in the mobile app	8
F13	As a Mobile App, I need an API to fetch all approved receipts, so that users can view their reimbursement history	Allows users to access their reimbursement records	3
F14	As a Mobile App, I need an API to create a new receipt and upload a receipt image, so that users can submit receipts directly from their phone	Enables mobile submission of receipts	3

ID	User Story	Purpose	SP
F15	As the System, I must require API keys for all API requests, so that only authorized apps can access receipt data	Ensures only trusted applications can access the system	8
F16	As the System, I want to fetch a list of bank payments, so that receipts can be matched to transactions	Enables matching receipts to real transactions	8
F17	As the System, I want to update the bank API to mark payments as approved or fraud, so that both systems stay consistent	Ensures bank transactions reflect the correct status	5

2.4.2 Non-functional requirements

- NF1 security API keys: The system must secure all API endpoints using API keys to ensure only authorized applications can access the data
- NF2 logging: The system must log all user actions
- NF3 usability: The user interface must be mobile-friendly so that salesmen can submit receipts while traveling
- NF4 performance: The system should load pages and receipt lists within a reasonable time
- NF5 Reliability: During runtime, the system should not lose data
- NF6 maintainability: The system must be built with a simple structure so that future developers can easily maintain and extend it
- NF7 availability: The system should be available during normal business hours without unexpected interruptions

2.4.3 Discussion on backlog structure

The backlog for this project has been developed based on the information gathered from the interview with the Accountant.

The requirements analysis for this project was carried out by identifying the needs of each user group identified earlier in the report. For every user type, we asked “If I were a user, what would I need from the system?” to use it as a guideline. This approach made it possible to create a clear set of functional requirements written as user stories.

From this process, a complete backlog of user stories was created and grouped into thematic categories. Grouping the stories into themes makes it easier to understand the structure of the system and reference related features together.

Each user story in the backlog has been roughly estimated using Planning Poker with story points based on the Fibonacci sequence (1, 2, 3, 5, 8, 13). These estimates are not precise time predictions but are based on intuition and expected workload. This gives the team an overview of which features are simple and which require more work.

The backlog also distinguishes between functional and non-functional requirements. Functional requirements describe what the system must do, such as submitting receipts, approving them, or managing users. Non-functional requirements describe how the system must behave, including things such as performance, usability, and security. Separating these categories makes it much clearer what is expected of the system and what the important aspects/requirements are.

2.4.4 Minimum Viable Product (MVP)

Based on the prioritized backlog, the Minimum Viable Product focuses on supporting the core concept of our system. For this first release, we include the following user stories:

- F1 - Salesmen can submit receipts
- F2 - Salesmen can see the status of their receipts
- F3 - Accountants can handle and validate receipts
- F4 - Managers can approve and reject handled receipts
- F7 - The system logs all actions on receipts

Together, these features provide a receipt workflow that can already be used in practice at GM. The remaining user stories are planned for later iterations once the MVP has been evaluated.

3 Part 2 - Implementation

In Part 2, we move from planning to implementation. The goal is to build the first working version of our receipt system. Based on user goals and requirements defined in the interview, we implemented an MVP that supports three fundamental processes, submitting a receipt, handling a receipt, and approving or rejecting a handled receipt. Our system is built around two main concepts, Users and Receipts

3.1 Users and Receipts

Receipts are the central object in the system. Each receipt contains a unique ID, amount, image path, submitter id, timestamp of submission, status, ID of users who handled/approved/rejected it, a handling log containing timestamped events.

Each receipt's handling log is saved in a list to ensure full traceability across the entire application, making it possible to always track who performed which action, when, and on which receipt.

Users represent the different roles in the organization, each with defined responsibility in the receipt workflow. All users shares common field such as id, username, email, name, password, and role. We currently support four roles:

The **Salesman** is responsible for submitting his own receipts. A salesman can create new receipts and view his previously submitted receipts. Once a salesman submits a receipt, the receipt will automatically receives the status PENDING, indicating that it is waiting for an accountant to review it. A salesman requires a manager when registering a new user.

The **Accountant** handles all receipts that are currently in the PENDING state. An accountant can view all pending receipts, perform validation and verification, and mark the receipt as HANDLED. Handling a receipt does not approve it, it only marks that the accountant has checked the details and confirmed that it is ready for approval.

The **Manager** is responsible for the final approval step. A manager can view all HANDLED receipts, approve or reject receipts, and submit receipts. A manager can never approve their own receipts, and as a fraud prevention we have decided to remove the feature "approve/reject" from his system when the receipt is his own. Approval changes the receipt status to APPROVED, while rejecting it changes the status to REJECTED.

Admin role here is just to have an overview, he can see all the registered users, all the receipts submitted and their status, and receipt handling log, such as who handled what or who approved/rejected what. For future improvement we could make it such that the admin or perhaps the manager can register/approve/delete users, as for now anyone can register as a manager, or accountant in our system.

3.2 Implementation details

For the implementation of this project we decided to use Python and Flask, combined with HTML/Jinja for the user interface. Flask provides the lightweight framework that fits well with our MVP, allowing us to focus on the core functionality, aligning with the Agile principle of simplicity, because we are able to focus on core functionality.

To organize the domain object and creating them, we implemented the **Factory Pattern**, **User_Factory**, and **Receipt_Factory**. This reduces duplication, ensures standardized creation, enforces correct initialization, performs validation (e.g. receipt amount ≥ 0) and allows us to use polymorphism when working with different user roles.

For internal system states, such as user role and receipt statuses, we implemented enums (UserRole and ReceiptStatus). This was done to improve type safety, readability, maintainability.

The data is stored in memory using Python lists located in database.py, we have a list for both users and receipts. This design choice avoids complexity and external databases, allowing us to focus on implementing core features. We also use a handling log inside each receipt (Receipt.handling_log) to track past events.

To separate the business logic we have created **User_Application** and **Receipt_Application**, so we don't need to rewrite or modify the business logic when changing the UI.

3.2.1 Implemented functionality

In this MVP, we implemented core functionality:

- **Create Receipt:** Salesmen and Managers can create and submit new receipts through the web UI. Each receipt is initialized with status "PENDING" and linked to the submitter
- **View Receipts:** Users can view receipts relevant to their role. Salesman can only view their own receipts, accountants can see all PENDING receipts, and manager can see all HANDLED receipts. Admin can see all receipts including their current status.
- **Detailed View Receipt:** A detail page shows full information about a single receipt, including submitter id, amount, timestamp and status.
- **Handle Receipt:** Accountants can review PENDING receipts and change the status to HANDLED.
- **Approve/Reject Receipt:** Manager make final approval decision on HANDLED receipts. The system implements a fraud prevention mechanism so that the managers cannot approve or reject their own receipts.
- **Create User:** New users register through the UI with one of the four implemented roles. The system requires a salesman to specify a manager during registration. The roles are final and can not be changed in our current system.
- **Admin:** Admins have an overview page displaying all registered users, their submitted receipts, and handling log. The idea is to give admin more features in the future so he can be able to control everything regarding receipts and users.
- **Logging:** Every important state change for the receipt is logged for fraud investigation and GDPR compliance. Each event is logged with user ID, the change of state for the receipt and timestamp.

3.3 User Interface Design

The UI for our MVP is designed to be simple, clear, and mobile friendly. We chose a clean layout, large buttons, and minimal text to ensure that users can complete tasks quickly. Especially for the traveling salesmen. The interface focuses only on the essential user flows required for the MVP

The system includes the following essential screens:

Login Page: Authenticates users and determines their role.

Log in

Enter your information to access

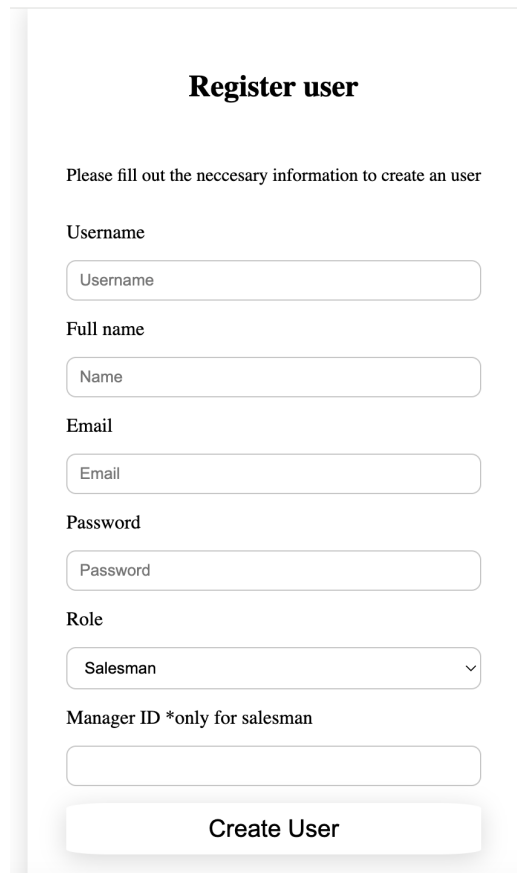
Username:

Password:

Log in

Figure 1: Enter Caption

Register Page: A form for creating a new user account, including username, name, email, password and role.

A registration form titled "Register user" with a subtitle "Please fill out the necessary information to create an user". It contains input fields for Username, Full name, Email, Password, and a dropdown for Role (set to Salesman). There is also a field for Manager ID with a note "*only for salesman". A "Create User" button is at the bottom.

Register user

Please fill out the necessary information to create an user

Username

Full name

Email

Password

Role

Manager ID *only for salesman

Create User

Figure 2: Register Page

Dashboard: A central page where each user is welcomed with their username and role, and presented with actions relevant to their responsibilities. The layout is the same across roles, but visible actions change accordingly.

Welcome, SALESMAN, salesman

What would you like to do?

[Upload New Receipt](#)

Receipts

ID	Amount	Status	Submitted by	Uploaded at	Action
1	199.5	PENDING	2	01-12-2025 20:37	

Figure 3: Salesman Dashboard

New Receipt Page: A simple form with fields for entering the amount and uploading a receipt image path.

Submit Receipt

Important: You MUST keep the original copy of the receipt

Amount in kr.

e.g 1459kr.

URL for receipt

e.g /image/receipt1

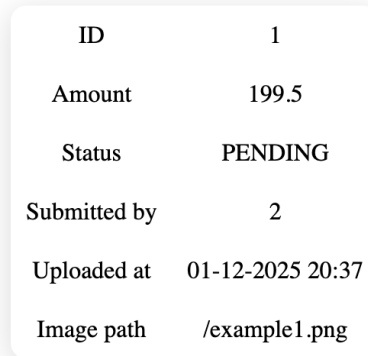
Submit receipt

Figure 4: New Receipt Page

Receipt Detail Page: A page displaying all available information about

a single receipt in a simple readable layout.

Receipt #1



ID	1
Amount	199.5
Status	PENDING
Submitted by	2
Uploaded at	01-12-2025 20:37
Image path	/example1.png

Figure 5: Receipt Details

We intentionally kept the UI minimal to support a fast MVP implementation, reduce cognitive load, and align with the stakeholder's need for a straightforward and efficient workflow

3.3.1 Demonstration video

The demonstration video for the full user interface can be found here.

Video link:

<https://drive.google.com/file/d/1CbeKxiT0x160E2zGfIdhHbqM6PXI1KcT/view?usp=sharing>

3.4 UML

The UML diagram in Figure 6 shows a domain model for the receipt handling system. Based on the two goals made being "Create and submit receipt" and "Approve receipt". The classes are divided into multiple domain entities User and Receipt, application of user and receipt, factories of them and a in-memory Database. User contains common data for all users, like their ID, name, email and so on. Where it also outlines all the authorization rules such as:

- `is_salesman()`, `is_accountant()`, `is_manager()`, `is_admin()` - a check on the given roles.

- `can_manage_users()`, `can_view_logs()`, `can_view_all_receipts()` - admin capabilities.
- `can_submit()`, `can_handle()`, `can_approve()`, `can_reject()`, `can_view()` - some permission checks used for the flow.

That means the User can specialize in 4 roles, admin, accountant, salesman (here a manager ID is needed for to work) and Manager. Using a inheritance based hierarchy should allow shared behaviour in User while also making sure that each role has its own capabilities.

The Receipt class stores the financial data and the status state: `id`, `amount`, `image_path`, `submitter_id`, `date`, `status` of the receipt, `handled_by_id`, `approved_by_id`, `rejected_by_id` and `handling_log` (used to append every action made such as approval, rejection with `add_event(message)`). As seen in the diagram a User can submit Receipt objects and that receipt is then processed by the application layer. The application layer contains operations on the domain objects. `UserApplication` maintains a list of User objects and provide some operations like `get_all_users()`, `validate_user(username, password)`, `get_user(user_id)` and `create_user(username, name, email, password, role, manager_id=None)` that will return the role (admin, salesman, accountant, manager) based on what role was chosen. `ReceiptApplication` contains the list for receipts and list of the Users, its operations are the main use case for the handling of receipts, Creation of new receipts are done with `receipt-factory`, with some validation like amount must be positive, while other level transitions like from `PENDING` to `HANDLED` to `APPROVED/REJECTED` are done in `ReceiptApplication`.

For the relationship between `ReceiptApplication` - `Receipt` and `UserApplication` - `User`, we have decided to use an aggregation. `ReceiptApplication` holds a reference to the list of Receipts in the database with the `self.repo` variable. It uses the `ReceiptFactory` to create new receipts, queries them and manipulates receipts by calling `handle()` and changing the status. We think an aggregation represents this the best for the following reasons: A dependency usually leads to a temporary or one-time use. This relationship is long-lived and thus we feel that it is not a dependency. While this may technically also be an association, we feel that an association is a bit too general, and does not cover the true relationship between the classes, where `ReceiptApplication` manages and contains one or many `Receipt` objects.

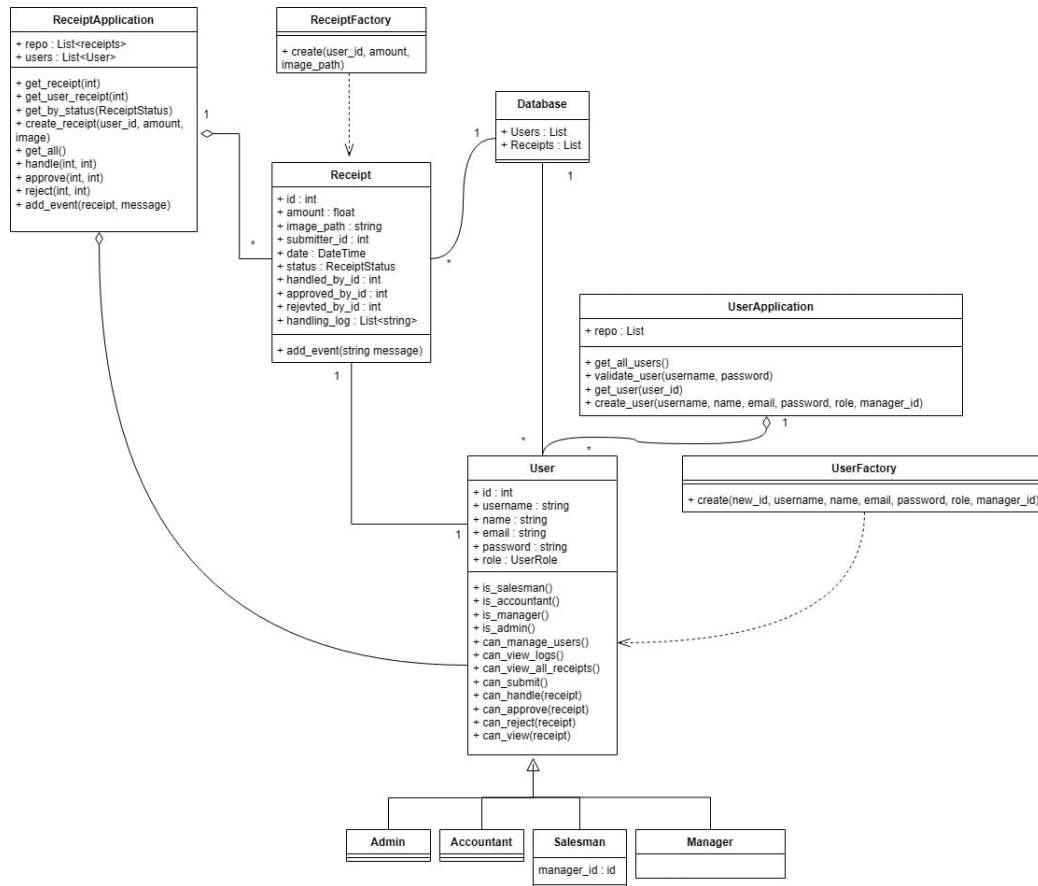


Figure 6: Class diagram

3.5 Sequence Diagram - Creating a Receipt

The sequence diagram shows the workflow when a salesman creates and submits a new receipt. After the User fills in the form for a receipt, the browser will then send a HTTP request to the app.py flask that checks if the user is allowed to submit the receipt and then forwards it ReceiptApplication. A method in ReceiptApplication is called via a method from ReceiptFactory to create this receipt with a new ID and sets the initial status as PENDING. The new receipt gets appended to the receipts list in models. When created we return to app.py flask where we get the conformation that is created and we get redirected to the dashboard. We make a GET request for it and this leads us to the app.py where we fetch the list of matching receipts in the database through ReceiptApplication. The result is then returned to the browser which will display the updated list of receipts.

This sequence shows how our layered architecture cleanly collaborates

and separates UI, application logic, domain logic, and data storage.

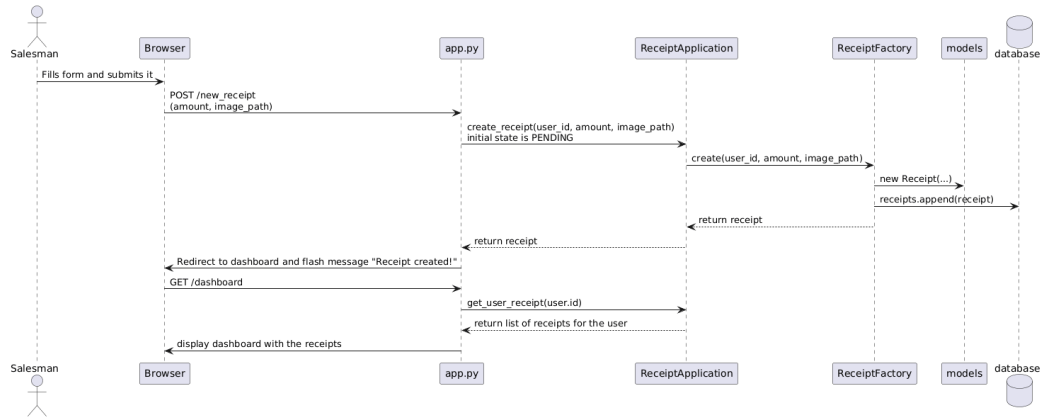


Figure 7: Create and submit sequence diagram

3.6 Sequence Diagram - Manager Approves a Receipt

This diagram describes the approval workflow when a Manager approves a handled receipt. The idea here is to demonstrate the system's business rules, including role restrictions and consistent state transitions.

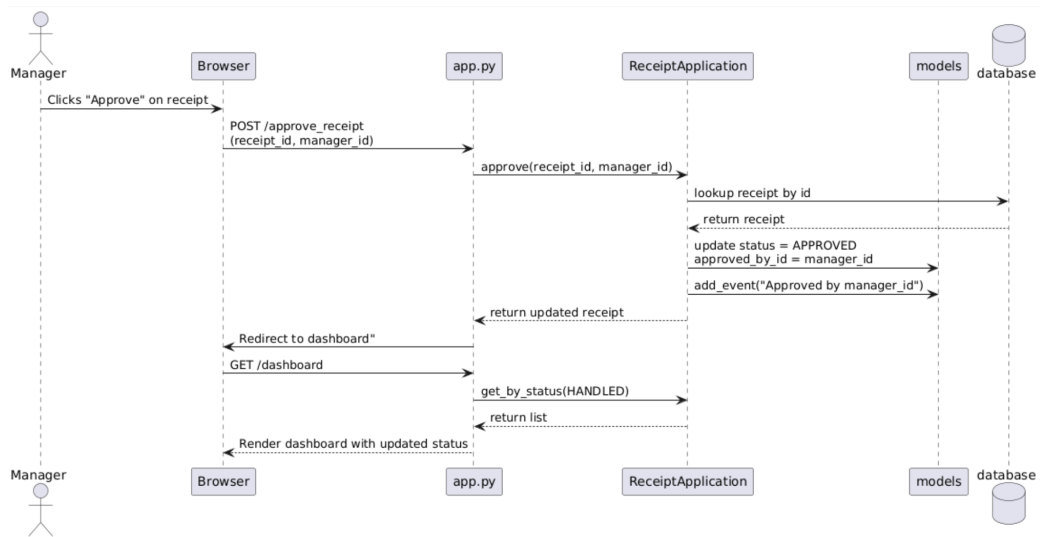


Figure 8: Approve Receipt sequence diagram

4 System Architecture - C4 Model

4.1 C4 Model

For the C4 model, we will present each level of abstraction to show the system from different perspectives, starting with the highest level System Context Diagram

4.1.1 System Context Diagram

In the C4 model, a system context diagram provides a high level overview of the system and its environment. It shows the software system as a single unit at the center, surrounded by the different user groups and the external systems it interacts with. At this level, we only focus on the people and systems that have a direct relationship with the system, without describing internal structure or details.

The System Context Diagram for our Receipt Management System is as following:

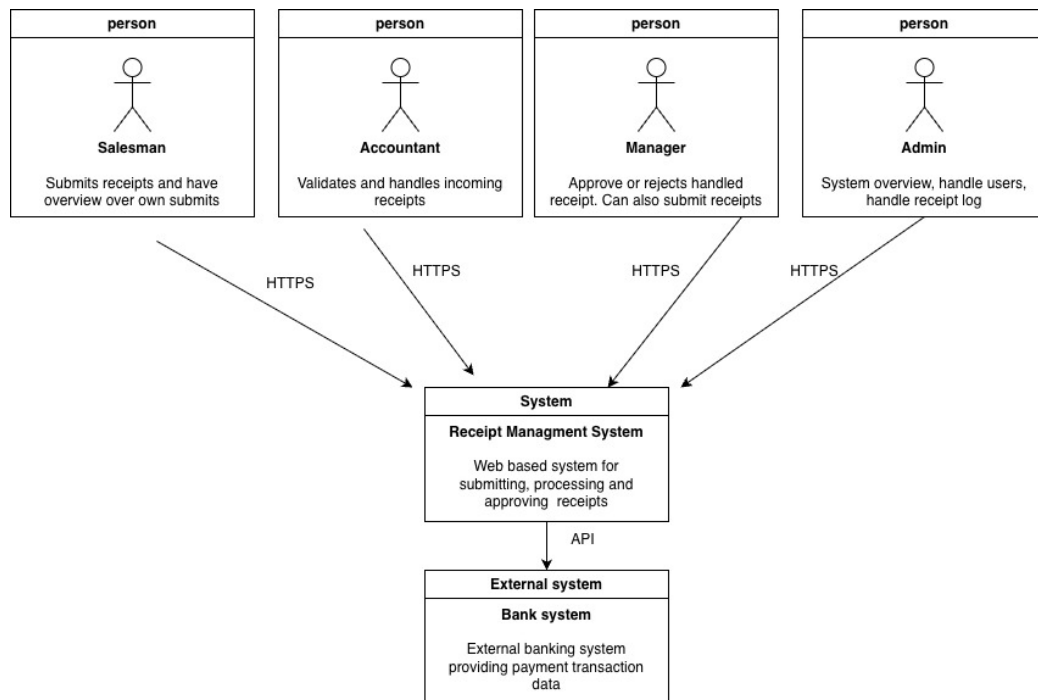


Figure 9: Level 1 of C4 model

This diagram illustrates the four primary user roles interacting with the Receipt Management System, as well as the external Bank System for future

integration.

4.1.2 Container Diagram

Once we understand how the system fits into its environment at a high level, the next step in the C4 model is to zoom inside the system using a Container Diagram. A container represents an executable application or a data store, such as a server web application, a mobile app, or a database.

For our Receipt Management System, the container diagram identifies two main containers, a Flask based web application that handles HTTP requests, business logic and HTML rendering, and an in memory data storage that temporarily stores users and receipts during runtime.

The Container Diagram is as following:

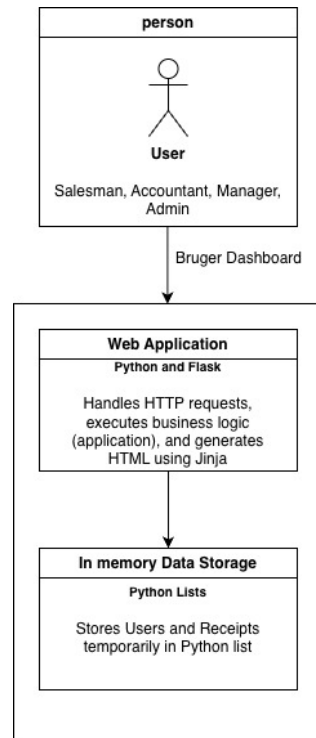


Figure 10: Level 2 of C4 model

4.1.3 Component Diagram

The Component Diagram represents the third level of the C4 model and provides a deeper look inside the Flask Web Application container. Instead of showing only the major applications, this level highlights how the container is structured and how responsibilities are organised.

At this level, we focus on understanding how the container's pieces work together, how responsibilities are separated, and how each component contributes to overall behaviour of the system without going all the way down to code details.

The Component Diagram is as following:

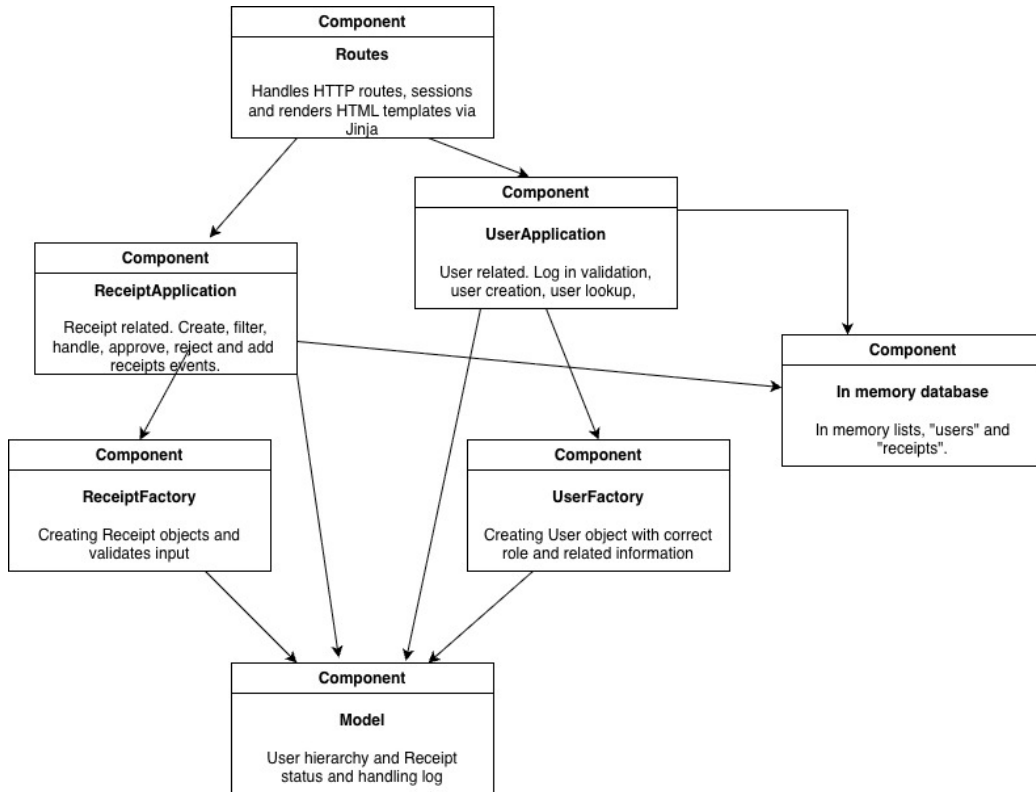


Figure 11: Level 3 of C4 model

Each component in the diagram represents a responsibility inside the Flask application.

4.1.4 Code Diagram

This level is usually not required, and would not add significant value to our system. Level 4 focuses on code level details, but most of this information is already visible directly in the source code.

At Level 4 diagram could be useful if we needed to visualise the implementation of complex component. In our case, the code is not at that level of complexity, and the structure is readable enough to serve as its own docu-

mentation. Therefore, creating a Level 4 diagram would not make sense in our case.

4.2 3 Layered Architecture

Our system is structured using a classic Three Layer Architecture, where each layer has clear responsibilities. This separation improves maintainance, redability, and testability.

1. **Presentation Layer** The presentation layer contains all Flask route handles, HTML/Jinja templates and user interaction. This layer consists of HTTP requests and responses, managing user session, and rendering HTML templates. The purpose is to act as the interface between the user and the underlying application logic.

2. **Application Layer**

This layer contains the business logic of the system. This layer coordinates the flow between the UI and the data itself. In our implementation we have included two applications, **User_Application** and **Receipt_Application** that are responsible for everything regarding user and receipts, such as user validation, user creation, receipts creation, filtering receipts and more. This is done with methods like `validate_user()`, `create_user`, `approve()`, `reject()` etc. This layer also contains the rules regarding salesman, accountant and manager and is easily testable.

3. **Data Layer**

The data layer stores and manages all domain objects. In this assignment, the data are stored in memory. To do this, we made `database.py` which contains lists of users and receipts. We have domain models for User, Salesman, Accountant, Manager, and Receipt.

We also have factories that create new domain objects in the correct state. Since all data are stored in memory, the system does not persist data between restarts. This is okay for now, but would not be sufficient for a deploy ready version of the project.

`UserApplication` and `ReceiptApplication` helps decouple business logic, and provides a single place for validation and status transitions. We also in-memory python lists in the `database.py` file for persistent storage. This is simple to do and does not require an external DB configuration, but it does come with some drawbacks. Firstly, this is not thread-safe and a proper database (like PostgreSQL) would fix this and should be used for a real deployment. We also can't enforce any constraints (keys, not null etc). Querying the database is also more efficient in a proper database.

4.3 Renzel's Distribution Patterns

Based on the distribution patterns described in Renzel's article, our system does not directly implement any fully distributed architecture, because our solution runs on a single Flask server with in memory data. However, if we compare our design to Renzel's pattern, our architecture mostly resembles the Distributed Presentation Architecture.

In this pattern, the presentation layer is split between client and server. The front end presentation runs on the client, which is the browser. It receives HTML from the server, renders it to the user, handles form input, and recognizes user action like "Approve Receipt" or "Handle Receipt".

The back end presentation runs on the server. It uses Jinja templates to generate HTML pages. For each user action, the server receives a request with data such as receipt ID and user ID, processes the request through the application kernel, and then generates a complete new HTML page so send back to the client.

4.4 Agile Principles

Our system architecture aligns with several of the Agile Principles.

Early and continuous delivery of valuable software

The chosen layered architecture allows us to deliver the system in stages and parallel work. The UI, business logic, factories and application services (like ReceiptApplication) are decoupled. The separation makes it possible for us to update the UI without touching the business logic. We can extend needed services without changing templates and add new features.

Welcoming changing requirements

Our design is open to change. We follow separation of concerns, and our code follows SOLID. This means future modifications have minimal impact on existing code. For example, adding a new user role only requires creating a subclass and updating the UserFactory, no changes to unrelated code (open-closed principle). The different subtypes (Manager, Salesman, etc.) can also be substituted for the base User type, since we have consistent checks with methods like `can_view_logs()` in the base class (LSP).

Deliver working software frequently

Using in memory storage instead of a full database reduces complexity and allows faster implementation of a Minimum Viable Product (MVP). This enables us to build core functionality quickly, iterate on upcoming feedback early. Once required, the Data Layer could be replaced with a real database without major changes.

We Are Agile!

Although we apply several Agile principles in our architecture and implementation, the nature of the assignment prevents us from following the Agile process in a realistic way. This project is based on a fixed interview text with fictional stakeholders, and we cannot collaborate with them. In real Agile development, developers and stakeholders work together continuously, clarify misunderstandings, requirements, and adjust plans based on feedback. In our case, the stakeholder information is static.

Because of this, our process ends up being closer to a Waterfall model than true Agile. This makes the project feel a bit more Agile inspired rather than fully Agile.

4.5 Unit Testing

We implemented unit tests for three classes in the system, the domain model in `models.py`, the user application layer, and the receipt application layer.

The tests focus on role based permissions and business rules, such as ensuring that a salesman can only view their own submitted receipts, a manager cannot approve their own receipt to prevent fraud, and that rejecting or approving a receipt updates the receipts states correctly.

We also validate that the application layer coordinates the receipt flow as intended. For example, our tests verify that a receipt cannot be approved before it has been handled by an accountant.

In addition, we test the creation of new objects, including creating users through `UserApplication.py` and generating new receipts through `ReceiptApplication.py`. This ensures that all objects are initialized in a valid state and stored correctly in the in memory database.

Some areas are not fully covered by tests, and a few edge cases remain for future development. Examples include invalid receipt IDs, attempts to handle receipts more than once, and more detailed error scenarios. These cases do not impact the core user flows in the current MVP but would be important for a production ready version.

In future test coverage, we could extend our tests to include Flask routes, HTML rendering, and additional error handling.

4.5.1 Coverage

Overall, the test suite provides solid coverage of the most important system behaviors, and it contributes positively to maintainability. For the current MVP stage, this level of coverage is sufficient, as the critical business logic in the domain layer and application layer is tested.

However, for a system that is ready to be deployed we would require additional tests, such as more detailed validation of user roles, negative tests involving invalid data or missing permissions, test of unsupported behavior, and integration tests covering Flask routes and full HTTP flows. Performance tests would also become relevant in case of scaling.

As the system evolves, the test suite can be extended accordingly to ensure that new features behave as intended and that existing functionality remains stable.

4.5.2 Unit Test Coverage Report

To evaluate the quality of our tests, we analyzed how much of the system's behavior is executed during testing. According to standard testing guidelines, code coverage is not a goal in itself but a tool for revealing untested code and missing behavior. It helps highlight which parts of the system we rely on but have not validated through tests.

Our coverage evaluations follows a structured approach where we map missing coverage back to specific features and rethink the test design, rather than writing trivial tests just to increase the percentage.

Coverage Results

To evaluate the reliability of our tests, we generated a code coverage report using pytest-cov. The following figure shows the coverage summary for the project:

===== tests coverage =====				
coverage: platform darwin, python 3.10.10-final-0				
Name	Stmts	Miss	Cover	Missing
app.py	160	160	0%	1-225
application/receipt_application.py	63	17	73%	27-31, 35-39, 55, 58, 70, 73, 85, 88, 91
application/user_application.py	20	4	80%	9, 19-21
database.py	3	0	100%	
enums.py	11	0	100%	
factories/receipt_factory.py	11	1	91%	8
factories/user_factory.py	17	6	65%	13-16, 18, 21-22
models.py	75	14	81%	17, 29, 32, 35, 38, 48, 54-58, 64-67
tests/test_models.py	33	0	100%	
tests/test_receiptApplication.py	47	0	100%	
tests/test_userApplication.py	19	0	100%	
TOTAL	459	202	56%	
===== 14 passed in 0.11s =====				

Figure 12: Tests Coverage

Based on the report, the core logic of the system achieves good overall coverage:

models.py: 81% - Most business rules related to roles, permissions, and receipt transitions are exercised by tests.

user_application.py: 80% - Input validation, login logic, and user creation through the factory are covered

receipt_application.py: 73% - The main receipt workflow, create -> handle -> approve -> reject, is covered, although a few conditional elements remain untested.

Flask routes and app.py: 0% - This is expected. Unit tests focus on business logic, not HTML rendering or HTTP routing.

Total project coverage: 56%

For an MVP, this level of coverage is appropriate. The part of the system that contain real business rules, the domain layer and applicatoin layer, are well covered. These layers handle permission checks, workflow transitions, and input validation, which are the most critical features in terms of correctness and saftey.

The main uncovered areas include, Flask routes, which can be difficult to do with unit test, negative/edge cases such as invalid receipt IDs, repeated handling attempts, and make sure permission paths are correctly implemented.

4.6 Mobile API

Mobile API This part of the project was to make a specification of a mobile API, which is to allow external smartphones to interact with the backend of a program. For this we used OpenAPI, which is a standard way to describe HTTP-based REST APIs. This does not implement the API but is just a formal way to describe the endpoints, request formats, responses, security and data. Ensuring that every party can agree to begin the development if it was a real project setting. The full specification made can be seen in the appendix C.

REST and Richardson Maturity Model Level 2

The API was focused around using the REST principles and had to reach level 2 in the Richardson Maturity Model (RMM). RMM is basically a way to show how well the API makes use of the REST principles. For that there are 4 levels normally, from 0 to 3, but for this project only up to level 2 is specified. To reach level 2 we have a few requirements that need to be met, and we will skip over level 0 as that is just using HTTP as a transport system for interactions. This usually means one single endpoint and the same HTTP method for everything. Level 1 and level 2 are the more important steps. Level 2 requires:

1. Resource URLs

Starting for the requirement of level 1, here we get to resources, so instead of making all requests to a single endpoint, each resource gets its own URL. So, the way of doing this is by getting the URL-like path to each distinct resource:

- /receipts
- /receipts/approved
- /receipts/receiptId/documents

2. Correct use of HTTP verbs

For the level 2 requirements we need to work with HTTP verbs. So, for wanting a list of information or reply with some information we use verbs, like GET and POST, there are multiple other verbs, but this project made only use of these two.

- GET for retrieving receipts
- POST for creating receipts
- POST with multipart/form-data for uploading documents

3. Meaningful HTTP status codes

After using the HTTP verbs with it naturally comes some response, that is these codes. If all goes well, we will get a response of 200 or 201 in this case to showcase that the HTTP request was a success. But just as important as a success response, when an error appears we also get a code to showcase it. Having the correct response codes for when something occurs during process is important for making the program easy to work with. Therefore, instead of using some return code of 200 and include an error response, in level 2 you explicitly use some kind of error response of your choice, but normally it would be a non-2xx response.

- 200 OK for successful read operations
- 201 Created for successful creation/upload
- 400 Bad Request for invalid input
- 401 Unauthorized for missing/invalid API key
- 500 Internal Server Error for unexpected failures

And with the use of HTTP verbs and HTTP response codes you would meet the requirements for a level 2 Richardson Maturity Model. API Key Security All endpoints are made with the use of API keys. API keys are an authentication mechanism for the HTTP verbs to work on requests. What it basically does is that before the mobile can interact with the backend it must be authenticated and that is where the API key does its job. All the requests that can be made must include this key to ensure that no unauthorized calls can occur on an external mobile. This also makes it easy to take away the key if needed from a device. The API-key in this program is defined under securitySchemes and then applied globally to each request to ensure they are all authorized to work.

Endpoint and Model Design

GET /receipts

Returns a list of all receipts, no matter the status. By using GET it is read-only, so no modification can be made here.

GET /receipts/approved

It is essentially the same thing as the GET /receipts, but here we just add a small filter to only show those that are approved.

POST /receipts

This is used for the creation of a new receipt. Here it is made so the client only has to specify two fields which are amount and description. The rest of the fields such as ID and status will be provided by the server itself. This is to ensure that the client does not have the ability to manipulate sensitive fields as it can be abused, for example by just approving the client's own receipts.

POST /receipts/receiptId/documents

Uploads a document (e.g., a photo of the receipt). It uses multipart data as it's typically used for requests that involve file uploads. The response is made so that it returns the updated receipt so the client immediately can see the new state.

Summary

The API is designed to be simple, secure, and compliant with the REST principles. Using resource URLs, correct HTTP verbs, good status codes, and an API-key security all together satisfy the specification we are looking for and the requirements to abide to a Level 2 RMM API. We then go over the specification where we defined how the mobile API can create receipts, fetch all of them or filter with only approved, upload documents while making sure that the server handles the sensitive fields such as ID and status.

4.7 Cyclomatic Complexity

When writing projects one would naturally strive for the code to be functional and good, but just as important for it to be simple and understandable. Because if your code is only readable by you and not anyone else, then the code is not really that useful. A simple way to measure the complexity of a piece of code is the Cyclomatic Complexity (CC). It is based on the number of independent decision paths through the program. Meaning the more branching, so stuff like if, while, for, and, or and so on, make it hard to test and maintain. So to ensure that we have some quality architecture and maintainability we calculate CC. We want the number to be low, since if a method had around 5 CC, it means you generally need 5 unit tests to make sure each of those paths are working as they should. So there is really no set amount that is considered a rule, but keeping it low enough is good practice. Now to go through each method and class. The highest amount does appear in flask controller, but that is expected, since it has multiple user-role checks and flow requests. While the business logic classes have very low 1-4 CC, since its quite linear and simple. Overall, the program is quite maintainable and the even when complexity is quite high, it is located in the appropriate layers. Here is the complete list over each class and its methods CC:

receipt_application.py

- `__init__` = 1
- `get_receipt` = 2
- `get_user_receipt` = 2
- `get_by_status` = 2
- `create_receipt` = 1
- `get_all` = 1

- handle = 4
- approve = 5
- reject = 6
- add_event = 1

Total CC for ReceiptApplication: **23**

user_application.py

- __init__ = 1
- get_all_users = 1
- validate_user = 3
- get_user = 2
- create_user = 3

Total CC for UserApplication: **10**

receipt_factory.py

- create = 2

Total CC for ReceiptFactory: **2**

user_factory.py

- create = 6

Total CC for UserFactory: **6**

app.py (Flask routes + helpers)

- index = 2
- register = 9
- login = 6
- dashboard = 5
- admin_dashboard = 3
- new_receipt = 4
- view_receipt = 5
- handle_receipt = 4
- approve_receipt = 4
- reject_receipt = 4
- logout = 2
- get_current_user = 2

Total CC for app.py: **44**

models.py

User class

- is_salesman = 1
- is_accountant = 1
- is_manager = 1
- is_admin = 1
- can_manage_users = 1
- can_view_logs = 1
- can_view_all_receipts = 1
- can_submit = 2

- can_handle = 1
- can_approve = 4
- can_reject = 4
- can_view = 4

Total CC for User class: **23**

Manager, Accountant, Salesman, Admin classes

- Each constructor = 1

Total CC for subclass constructors: **4**

Receipt class

- __init__ = 1
- add_event = 1

Total CC for Receipt: **2**

HTML templates

CSS does not use any logic that counts as CC, but if HTML uses something like jinja format: "{% if user.can_submit() %}", meaning it uses "if" logic, thus it counts as complexity.

- receipt_detail.html = 1
- register.html = 1
- new_receipt.html = 1
- login.html = 1
- dashboard.html = 7
- base.html = 3
- admin_dashboard.html = 8

Total CC for templates: **22**

4.8 Bank integration

A video demonstration of the bank integration a proof-of-concept shows that retrieving customers and their payments from the bank API, update the payments with a approved or reject button and confirmation via the response codes to see it being called. The video can be seen with this link.

Video link:

https://drive.google.com/file/d/1LBD_DpG0vR4mLYTy2qIDgBhBKrvPudi6/view?usp=sharing

A Credit - Who Wrote What

- **moami14:** 3.1, 3.2, 3.3, 4.1, 4.2, 4.4, 4.3, 4.5
- **tkhan23:** 2.2, 2.3, 2.4, 3.4, 4.2, 4.4
- **jogas23:** 3.4, 3.5, 3.6, 4.6, 4.7, 4.8
- **kaami23:** 2.1, 2.2, 2.3, 2.4, 3.2, 4.5
- **General collaboration:** Although each section has a primary author, the group worked collaboratively across the entire report. Many parts were discussed, reviewed, and refined together, and ideas were shared to ensure consistency across the report, so at the end we all had something we agreed on.

B System Source Code

B.1 enums.py

```
1 from enum import Enum
2
3 class ReceiptStatus(Enum):
4     PENDING = "PENDING"
5     HANDLED = "HANDLED"
6     APPROVED = "APPROVED"
7     REJECTED = "REJECTED"
8
9 class UserRole(Enum):
10     SALESMAN = "SALESMAN"
11     ACCOUNTANT = "ACCOUNTANT"
12     MANAGER = "MANAGER"
13     ADMIN = "ADMIN"
```

B.2 database.py

```
1 from models import Manager, Salesman, Accountant, Admin
2
3 users = [
4     Manager(1, "manager", "manager", "manager@live.dk",
5             "manager"),
6     Accountant(3, "accountant", "accountant",
7               "accountantlive.dk", "accountant"),
```

```

6     Salesman(2, "salesman", "salesman", "salesman@live.dk",
7               "salesman", manager_id=1),
8     Admin(4, "admin", "admin", "admin@live.dk", "admin")
9 ]
9 receipts = []

```

B.3 models.py

```

1 from datetime import datetime
2 from enums import ReceiptStatus, UserRole
3
4
5
6 class User:
7     def __init__(self, id, username, name, email, password,
8                 role):
9         self.id = id
10        self.username = username
11        self.name = name
12        self.email = email
13        self.password = password
14        self.role = role
15
16    def __repr__(self):
17        return f'<User id={self.id} username={self.username} role={self.role}>'
18
19    def is_salesman(self):
20        return self.role == UserRole.SALESMAN
21
22    def is_accountant(self):
23        return self.role == UserRole.ACCOUNTANT
24
25    def is_manager(self):
26        return self.role == UserRole.MANAGER
27
28    def is_admin(self):
29        return self.role == UserRole.ADMIN
30
31    def can_manage_users(self):
32        return self.is_admin()
33
34    def can_view_logs(self):
35        return self.is_admin()
36
37    def can_view_all_receipts(self):

```

```

38         return self.is_admin()
39
40     def can_submit(self):
41         return self.is_salesman() or self.is_manager()
42
43     def can_handle(self, receipt):
44         return self.is_accountant() and receipt.status ==
            ReceiptStatus.PENDING
45
46     def can_approve(self, receipt):
47         if not self.is_manager():
48             return False
49         if receipt.submitter_id == self.id:
50             return False
51         return receipt.status == ReceiptStatus.HANDLED
52
53     def can_reject(self, receipt):
54         if not self.is_manager():
55             return False
56         if receipt.submitter_id == self.id:
57             return False
58         return receipt.status == ReceiptStatus.HANDLED
59
60     def can_view(self, receipt):
61         if self.is_salesman():
62             return receipt.submitter_id == self.id
63
64         if self.is_accountant() or self.is_manager():
65             return True
66
67         return False
68
69
70     class Manager(User):
71         def __init__(self, id, username, name, email, password):
72             super().__init__(id, username, name, email,
                password, UserRole.MANAGER)
73
74     class Accountant(User):
75         def __init__(self, id, username, name, email, password):
76             super().__init__(id, username, name, email,
                password, UserRole.ACCOUNTANT)
77
78
79     class Salesman(User):
80         def __init__(self, id, username, name, email, password,
            manager_id):
81             super().__init__(id, username, name, email,
                password, UserRole.SALESMAN)

```

```

82         self.manager_id = manager_id
83
84 class Admin(User):
85     def __init__(self, id, username, name, email, password):
86         super().__init__(id, username, name, email,
87                             password, UserRole.ADMIN)
88
89 class Receipt:
90     def __init__(self, id, amount, image_path,
91                 submitter_id):
92         self.id = id
93         self.amount = amount
94         self.image_path = image_path
95         self.submitter_id = submitter_id
96
97         self.date = datetime.now()
98
99         self.status = ReceiptStatus.PENDING
100
101         self.handled_by_id = None
102         self.approved_by_id = None
103         self.rejected_by_id = None
104
105         self.handling_log = []
106
107
108     def add_event(self, message):
109         self.handling_log.append(f"{datetime.now()}: {message}")

```

B.4 user_application.py

```

1 from factories.user_factory import UserFactory
2
3 class UserApplication:
4     """
5     Applicato in layer responsible for managing, authenticate
6     and creating users
7     """
8
9     def __init__(self, users):
10         self.repo = users
11
12     def get_all_users(self):
13         return self.repo

```

```

13
14     def validate_user(self, username, password):
15         users = [x for x in self.repo if x is not None]
16
17         for user in users:
18             if user.username == username and user.password
19                 == password:
20                 return user
21
22     def get_user(self, user_id):
23         """Return a user by a given user ID"""
24         for x in self.repo:
25             if x.id == user_id:
26                 return x
27
28     def create_user(self, username, name, email, password,
29                     role, manager_id=None):
30         new_id = len(self.repo) + 1
31         user = UserFactory.create(new_id, username, name,
32                                   email, password, role, manager_id)
33         self.repo.append(user)
34         return user

```

B.5 receipt_application.py

```

1 from enums import ReceiptStatus
2 from datetime import datetime
3 from factories.receipt_factory import ReceiptFactory
4
5
6 class ReceiptApplication:
7     """
8     Application layer for working with receipts.
9     Handles receipt creation and transitions between states
10    of receipts
11    """
12
13    def __init__(self, receipts, users):
14        self.repo = receipts
15        self.users = users
16
17    def get_receipt(self, receipt_id):
18        """Return a receipt with a given id"""
19        for x in self.repo:
20            if x.id == receipt_id:
21                return x

```



```

22
23 def get_user_receipt(self, user_id):
24     """Return all receipts submitted by user."""
25     result = []
26     for x in self.repo:
27         if x.submitter_id == user_id:
28             result.append(x)
29     return result
30
31
32 def get_by_status(self, status):
33     """Return all receipts with a given status"""
34     result = []
35     for x in self.repo:
36         if x.status == status:
37             result.append(x)
38     return result
39
40 def get_all(self):
41     """Return all receipts."""
42     return self.repo
43
44
45 def create_receipt(self, user_id, amount, image_path):
46     receipt = ReceiptFactory.create(user_id, amount,
47                                     image_path)
48     self.repo.append(receipt)
49
50     return receipt
51
52 def handle(self, receipt_id, accountant_id):
53     receipt = self.get_receipt(receipt_id)
54
55     if receipt is None:
56         raise ValueError("Receipt not found")
57
58     if receipt.status != ReceiptStatus.PENDING:
59         raise ValueError("Only PENDING receipts")
60
61     receipt.status = ReceiptStatus.HANDLED
62     receipt.handled_by_id = accountant_id
63     self.add_event(receipt, f"Handled by {accountant_id}")
64     return receipt
65
66 def approve(self, receipt_id, manager_id):
67
68     receipt = self.get_receipt(receipt_id)

```

```

69
70     if receipt.status != ReceiptStatus.HANDLED:
71         #Rejected receipt isn't handled again
72         raise ValueError("Cannot approve receipt that is
73                             not handled yet")
74
75     if receipt.submitter_id == manager_id:
76         raise ValueError("FRAUD PREVENTION! Manager
77                             cannot approve own receipt")
78
79     receipt.status = ReceiptStatus.APPROVED
80     receipt.approved_by_id = manager_id
81     self.add_event(receipt, f"Approved by {manager_id}")
82
83     return receipt
84
85 def reject(self, receipt_id, manager_id):
86     receipt = self.get_receipt(receipt_id)
87
88     if receipt is None:
89         raise ValueError("Receipt not found")
90
91     if receipt.status != ReceiptStatus.HANDLED:
92         raise ValueError("Cannot reject receipt that is
93                             not handled yet")
94
95     if receipt.submitter_id == manager_id:
96         raise ValueError("Managers cannot reject their
97                             own receipts")
98
99     receipt.status = ReceiptStatus.REJECTED
100     receipt.rejected_by_id = manager_id
101     self.add_event(receipt, f"Rejected by {manager_id}")
102
103     return receipt
104
105 def add_event(self, receipt, message):
106     receipt.handling_log.append(f"{datetime.now().strftime('%d-%m-%Y
107                                     %H:%M:%S')}: {message}")

```

B.6 receipt_factory.py

```

1 from models import Receipt
2 from database import receipts
3
4 class ReceiptFactory:
5     def create(user_id, amount, image_path):

```

```

6         if amount <= 0:
7             raise ValueError("The amount should be more than 0")
8
9         new_id = len(receipts) + 1
10        receipt = Receipt(new_id, amount, image_path,
11                           user_id)
12
13        return receipt

```

B.7 user_factory.py

```

1 from models import Salesman, Manager, Accountant, Admin
2
3 class UserFactory:
4
5     def create(new_id, username, name, email, password,
6               role, manager_id = None):
7
8         match role:
9             case "SALESMAN":
10                if manager_id is None:
11                    raise ValueError(f"Salesman {username} needs to have a manager.")
12
13                user = Salesman(new_id, username, name,
14                               email, password, manager_id)
15             case "ACCOUNTANT":
16                user = Accountant(new_id, username, name,
17                                email, password)
18             case "MANAGER":
19                user = Manager(new_id, username, name,
20                              email, password)
21             case "ADMIN":
22                user = Admin(new_id, username, name, email,
23                             password)
24
25        return user

```

B.8 app.py

```

1 from flask import Flask, render_template, request, session,
2   redirect, url_for, flash
3 from database import users, receipts
4 from application.user_application import UserApplication

```

```

4 from application.receipt_application import
    ReceiptApplication
5 from enums import ReceiptStatus
6
7 app = Flask(__name__)
8 app.secret_key = "secretkey"
9
10
11 receipt_application = ReceiptApplication(receipts, users)
12 user_application = UserApplication(users)
13
14
15 @app.route("/")
16 def index():
17     if "user" not in session:
18         return redirect(url_for("login"))
19     return redirect(url_for("dashboard"))
20
21 @app.route("/register", methods=["POST", "GET"])
22 def register():
23     if "user" in session:
24         return redirect(url_for("dashboard"))
25
26     if request.method == "POST":
27         username = request.form.get("username")
28         name = request.form.get("name")
29         email = request.form.get("email")
30         password = request.form.get("password")
31         role = request.form.get("role")
32         manager_id = request.form.get("manager_id")
33
34         if role == "SALESMAN":
35             if not manager_id:
36                 flash("Salesman must have a manager ID.")
37                 return render_template("register.html")
38
39             manager =
                user_application.get_user(int(manager_id))
40
41             if manager is None or not manager.is_manager():
42                 flash("Manager ID does not exist")
43                 return render_template("register.html")
44         else:
45             manager_id = None
46
47         user_application.create_user(username =
            username, name = name,
48             email = email, password = password, role = role,
                manager_id = manager_id

```

```

49         )
50
51
52         flash(f"User_{username}_created!")
53         return redirect("/login")
54     return render_template("register.html")
55
56 @app.route("/login", methods=["GET", "POST"])
57 def login():
58     if "user" in session:
59         return redirect(url_for("dashboard"))
60
61     if request.method == "POST":
62         username = request.form.get("username")
63         password = request.form.get("password")
64
65         user = user_application.validate_user(username,
66                                             password)
67
68         if user:
69             session["user"] = user.id
70             session["role"] = user.role.value
71
72             if user.is_admin():
73                 return redirect(url_for("admin_dashboard"))
74
75             return redirect(url_for("dashboard"))
76         else:
77             flash("Incorrect_username_or_password")
78             return redirect(url_for("login"))
79
80     return render_template("login.html")
81
82
83 @app.route("/dashboard")
84 def dashboard():
85     user = get_current_user()
86
87     if user is None:
88         return redirect(url_for("login"))
89
90     if user.is_salesman():
91         show_receipts =
92             receipt_application.get_user_receipt(user.id)
93     elif user.is_accountant():
94         show_receipts =
95             receipt_application.get_by_status(ReceiptStatus.PENDING)
96     elif user.is_manager():

```

```

95         show_receipts =
96             receipt_application.get_by_status(ReceiptStatus.HANDLED)
97     elif user.is_admin():
98         show_receipts = receipt_application.get_all()
99     else:
100         show_receipts = []
101
102     return render_template("dashboard.html", user=user,
103                           receipts = show_receipts)
104
105 @app.route("/admin")
106 def admin_dashboard():
107     user = get_current_user()
108
109     if user is None:
110         return redirect(url_for("login"))
111
112     if not user.is_admin():
113         return redirect(url_for("dashboard"))
114
115     all_users = user_application.get_all_users()
116     all_receipts = receipt_application.repo
117
118     return render_template("admin_dashboard.html",
119                           user=user, users=all_users, receipts=all_receipts)
120
121 @app.route("/new_receipt", methods=["GET", "POST"])
122 def new_receipt():
123     user = get_current_user()
124
125     if user is None:
126         return redirect(url_for("login"))
127
128     if not user.can_submit():
129         return redirect(url_for("dashboard"))
130
131     if request.method == "POST":
132         amount = float(request.form.get("amount"))
133         image = request.form.get("image_path")
134
135         receipt_application.create_receipt(user.id, amount,
136                                           image)
137         flash("Receipt created!")
138         return redirect(url_for("dashboard"))
139
140     return render_template("new_receipt.html", user=user)
141
142 @app.route("/receipt/<int:receipt_id>")
143 def view_receipt(receipt_id):

```

```

140     receipt = receipt_application.get_receipt(receipt_id)
141     user = get_current_user()
142     if user is None:
143         return redirect(url_for("login"))
144
145     if receipt is None:
146         flash("Receipt does not exist.")
147         return redirect(url_for("dashboard"))
148
149     if not user.can_view(receipt):
150         flash("You cannot view this receipt.")
151         return redirect(url_for("dashboard"))
152
153     return render_template("receipt_detail.html", user=user,
154                           receipt=receipt)
155
156 @app.route("/receipt/<int:receipt_id>/handle",
157           methods=["POST"])
158 def handle_receipt(receipt_id):
159     user = get_current_user()
160     receipt = receipt_application.get_receipt(receipt_id)
161
162     if not user.is_accountant():
163         flash("Only accountants can handle receipts.")
164         return redirect(url_for("dashboard"))
165
166     if not user.can_handle(receipt):
167         flash("You cannot handle this receipt.")
168         return redirect(url_for("dashboard"))
169
170     receipt_application.handle(receipt_id, user.id)
171     flash(f"Receipt {receipt_id} is now handled")
172     return redirect(url_for("dashboard"))
173
174 @app.route("/receipt/<int:receipt_id>/approve",
175           methods=["POST"])
176 def approve_receipt(receipt_id):
177     user = get_current_user()
178     receipt = receipt_application.get_receipt(receipt_id)
179
180     if not user.is_manager():
181         flash("Only managers can approve receipts")
182         return redirect(url_for("dashboard"))
183
184     if not user.can_approve(receipt):
185         flash("You cannot approve this receipt")
186         return redirect(url_for("dashboard"))

```

```

186
187     receipt_application.approve(receipt_id, user.id)
188     flash(f"Receipt_{receipt_id}_approved")
189     return redirect(url_for("dashboard"))
190
191
192 @app.route("/receipt/<int:receipt_id>/reject",
193           methods=["POST"])
194 def reject_receipt(receipt_id):
195     user = get_current_user()
196     receipt = receipt_application.get_receipt(receipt_id)
197
198     if not user.is_manager():
199         flash("Only_managers_can_reject_receipts.")
200         return redirect(url_for("dashboard"))
201
202     if not user.can_reject(receipt):
203         flash("You_cannot_reject_this_receipt.")
204         return redirect(url_for("dashboard"))
205
206     receipt_application.reject(receipt_id, user.id)
207     flash(f"Receipt_{receipt_id}_rejected")
208     return redirect(url_for("dashboard"))
209
210 @app.route("/logout")
211 def logout():
212     if "user" in session:
213         session.clear()
214         flash("You_have_been_logged_out")
215         return redirect(url_for("login"))
216
217     return redirect(url_for("login"))
218
219 def get_current_user():
220     user_id = session.get("user")
221     if user_id is None:
222         return None
223
224     return user_application.get_user(user_id)
225
226 if __name__ == "__main__":
227     app.run(debug=True)

```

C Mobile API Specification (OpenAPI)

openapi: 3.2.0
info:


```

title: ApI for mobile apps (DM571)

servers:
  - url: https://API-For-Mobile-Apps.com

components:
  securitySchemes:
    ApiKeyAuth:
      type: apiKey
      in: header
      name: SomeKey

  schemas:
    Receipt:
      type: object
      required: [id, amount, status]
      properties:
        id:
          type: string
        amount:
          type: number
        status:
          type: string
          enum: [pending, approved, fraud]
        description:
          type: string

security:
  - ApiKeyAuth: []

paths:
  /receipts:
    get:
      summary: Get all receipts
      security:
        - ApiKeyAuth: []
      responses:
        '200':
          description: List of receipts
          content:

```

```

    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/Receipt'
      example:
        - id: "58"
          amount: 9999999
          status: "fraud"
          description: "Transfer to other account"
        - id: "11"
          amount: 1200
          status: "approved"
          description: "Mcdonalds"
  '401':
    description: Invalid or missing API key
  '500':
    description: Server error

post:
  summary: Create a new receipt
  security:
    - ApiKeyAuth: []
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          properties:
            amount:
              type: number
            description:
              type: string
  responses:
    '201':
      description: Receipt created
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Receipt'

```

```

        example:
            id: "60"
            amount: 500
            status: "pending"
            description: "House rent"
    '401':
        description: Invalid or missing API key
    '500':
        description: Server error
    '400':
        description: Input validation failed

/receipts/approved:
get:
    summary: Get all approved receipts
    security:
        - ApiKeyAuth: []
    responses:
        '200':
            description: List of approved receipts
            content:
                application/json:
                    schema:
                        type: array
                        items:
                            $ref: '#/components/schemas/Receipt'
                    example:
                        id: "61"
                        amount: 150
                        status: "approved"
                        description: "Grocery shopping"
        '401':
            description: Invalid or missing API key
        '500':
            description: Server error

/receipts/{receiptId}/documents:
post:
    summary: Upload a document
    security:
        - ApiKeyAuth: []

```

```

parameters:
  - in: path
    name: receiptId
    required: true
    schema:
      type: string
requestBody:
  required: true
  content:
    multipart/form-data:
      schema:
        type: object
        properties:
          file:
            type: string
            format: binary
responses:
  '201':
    description: Document uploaded
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Receipt'
  '401':
    description: Invalid or missing API key
  '500':
    description: Server error
  '400':
    description: Input validation failed

```